Case Study III: Movie Recommender System

By

Nikitha Joshy

10/2/2025

"On my honor as an Aggie, I have neither given nor received unauthorized aid on this academic work."

This is entirely my own work or—if authorized by my instructor—the work of the team I was assigned to, except as documented in a bibliography or acknowledged below.

Signature * Nikitha Joshy

Artificial Intelligence (AI) Acknowledgement:
Google Gemini assisted with revisions to this report in the limitations and methods section. Copilot was used to assist with Python during model and dashboard creation with Streamlit.

Submitted to fulfill the requirement for Case Study 3
DAEN 400, Fall 2025
Department of Industrial & Systems Engineering
Texas A&M University

Executive Summary

This project was undertaken to build a custom Movie Recommender using Lenskit and the 32M dataset from MovieLens. The model employs collaborative filtering, content modelling, a popularity prior, and a hybrid scoring system to recommend 10 movies to a user based on how they rate 5 random movies. The model replicates a simpler version of the methods used by Netflix and Spotify. Limitations include reliance on basic metadata, a tendency toward mainstream picks, and diversity based only on content similarity. Recommendations for an improved model include adding user demographic data, public deployment with accounts to keep track of a user's ratings and an "already watched" control so movies that a user has not seen aren't recommended.

Table of Contents

Executive Summary 2
Table of Contents 3
List of Figures 4
1. Introduction 5
2. Methods 5
2.1 Model Creation 5
2.1.1 Content Modeling 5
2.1.2 Collaborative Filtering 6
2.1.3 Popularity Prior 6
2.1.4 Hybrid Blend & Diversity (MMR) 6
2.2 Streamlit UI 7
3. Results 7
4. Conclusions 10
4.1 How the Dashboard Works 10
4.2 Comparison to Existing Recommender Systems 11
4.2.1 Comparison to Netflix 11
4.2.2 Comparison to Spotify 11
5. Limitations 11
6. Recommendations 12
7. References 13

List of Figures

Figure 1. Complete Movie Recommender User Interface 7
Figure 2. Preferred Weights 8
Figure 3. Visual Weight Summary 9
Figure 4. Recommended Movie Interface 10

1. Introduction

Recommender systems have been a cornerstone of the internet for years, helping match users with things that align with their interests. Case Study III focuses on the use of LensKit for making a movie recommender system. LensKit was originally in Java but is now readily available in Python and it provides a set of user-friendly tools for "training, running, and evaluating recommender algorithms in a flexible fashion suitable for research and education." The dataset used in this case study is the 32M dataset which was "collected and made available rating data sets from the MovieLens web site" and it has "32,000,204 ratings and 2,000,072 tag applications across 87,585 movies."

Using both the 32M dataset and LensKit, the objective of Case Study III was to develop a custom movie recommender system with an interactive UI, in this case Streamlit was used. Existing recommender systems were also reviewed compared to the custom recommender which is an overly simplified version of existing systems like Netflix, Amazon, and Spotify.

2. Methods

2.1 Model Creation

To build a model which recommends movies to a user based on how they use rate some movies, there were 2 components: a model2.py and a streamlit app.py. Both files worked together to make a user friendly and interactive dashboard to recommend movies to users.

For the model, there are multiple factors that go into how a movie is recommended for users. The model uses content modeling (genres and tags), collaborative filtering (item-item KNN), popularity prior, hybrid blend, and diversity to suggest new movies to a user.

2.1.1 Content Modeling

For the content modeling aspect, term frequency and inverse document frequency (TF – IDF) were employed to build the features from genres and tags. In the dataset, tags just represent a few words or phrases that represent attributes about a specific movie. So, TF – IDF was used to make a user profile that averages the vectors of the 5 movies a user rates on Streamlit. Then those candidates are scored and used to recommend movies of similar content, and it works well even though a user is not rating many movies before getting suggestions. Content modeling is also a fast and lightweight option, which is great for a dataset of this size.

2.1.2 Collaborative Filtering

Directly from LensKit, item-item KNN was used for collaborative filtering to recommend movies that people with similar taste liked, regardless of the actual content. Collaborative filtering is a powerful method that helps find patterns in the data that may not be inherently clear to see, however there is a popularity bias. The item-item KNN itself makes a graph that connects the most similar items (movie) based on the ratings in the dataset and which users like what. So, each item is represented by a vector or ratings, and those ratings come from many users. Then the similarity between items is computed and the top movie for a user is recommended through a weighted average based on how they rated the 5 sample movies on the streamlit dashboard. Using item-item KNN gives stable similarities because more data means more interactions. It is also reasonable to assume that a user might like a specific movie because other people enjoy similar movies.

2.1.3 Popularity Prior

A popularity prior in the model makes sure that the movie recommender considers popular movies, which are movies that were rated highly by many users. The model calculates priors by multiplying the average rating a movie has by a shrunken version of the number of ratings that movie has in the data. Multiplying the (number of ratings +1) by log is what shrinks the value to make popularity have less of a severe impact on the final recommendations. A popularity prior was chosen to be used because it makes sure that movies with more good ratings would outrank good movies with less ratings in general. Popularity prior can also be adjusted to fit more complicated or fancier models.

2.1.4 Hybrid Blend & Diversity (MMR)

Hybrid blending was important for the movie recommender model because the collaborative, content, and popularity prior scores all have to be turned into one comprehensive score. All the components are put on the same scale after being min-max normalized because not all the scores are on the same scale, so weights are applied to them and renormalized. The weights can be changed by the user in Streamlit. So hybrid blending is an integral part of combining all the different components of the model to produce 10 final movie recommendations for a user. For diversity, MMR stands for maximal marginal relevance and its an algorithm that balances the relevance of movies and how different it is from other movies. In this model, the relevance comes from the hybrid score calculated and the similarity of two movies comes from the content modeling portion of the modeling. So MMR makes sure that movies that are similar are recommended, but not movies that are too similar to ones liked by the user. Diversity is an important part of the model to include so that while good movies are recommended, there is also a variety in the movies recommended, and gives users the opportunity to discover new movies.

2.2 Streamlit UI

Streamlit was chosen to be the dashboard for the Movie Recommender model because of multiple reasons that make it a good lightweight too. Streamlit fits well with Python, and no other language is needed to make the front-end. There are also built in tools within Streamlit that can be used to design and make a good intuitive interface for users, and Streamlit also allows caching. So in conclusion, Streamlit was chosen over other dashboard frameworks because of its simplicity, reactive workflow, session state and caching, and the way that it can visually integrate python tools easily.

3. Results

Figure **1** below shows the main dashboard of the Movie Recommender.

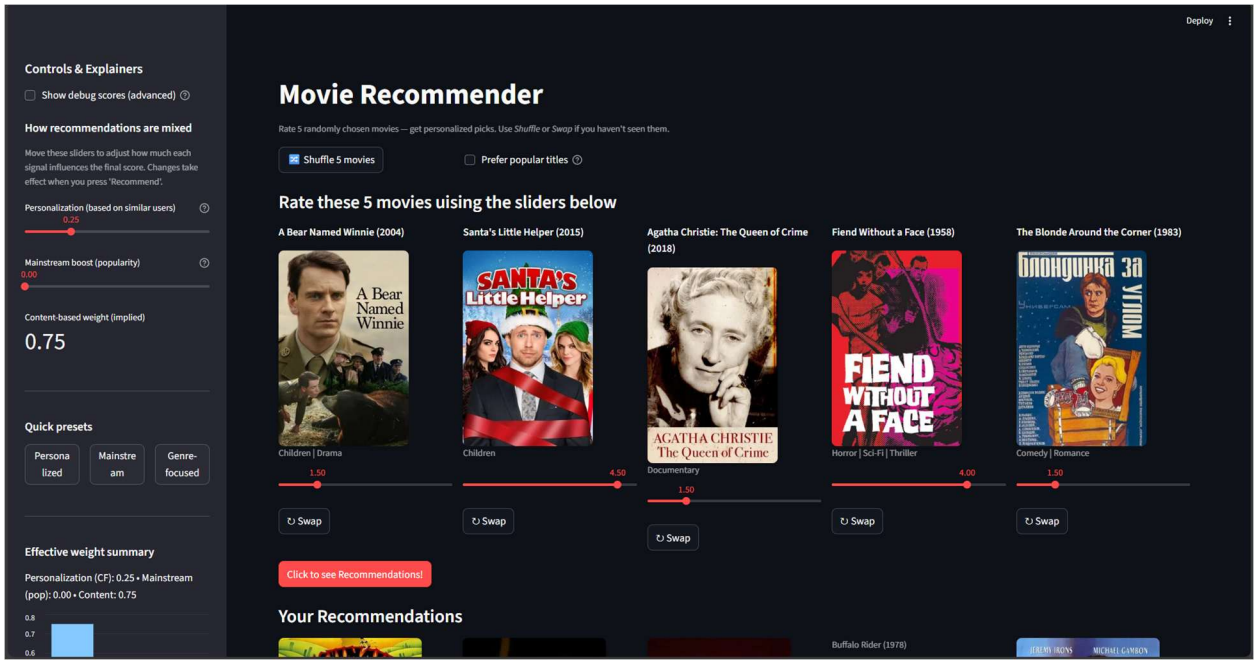


Figure 1. Complete Movie Recommender User Interface

Figure 2 below shows the sidebar of the Movie Recommender.

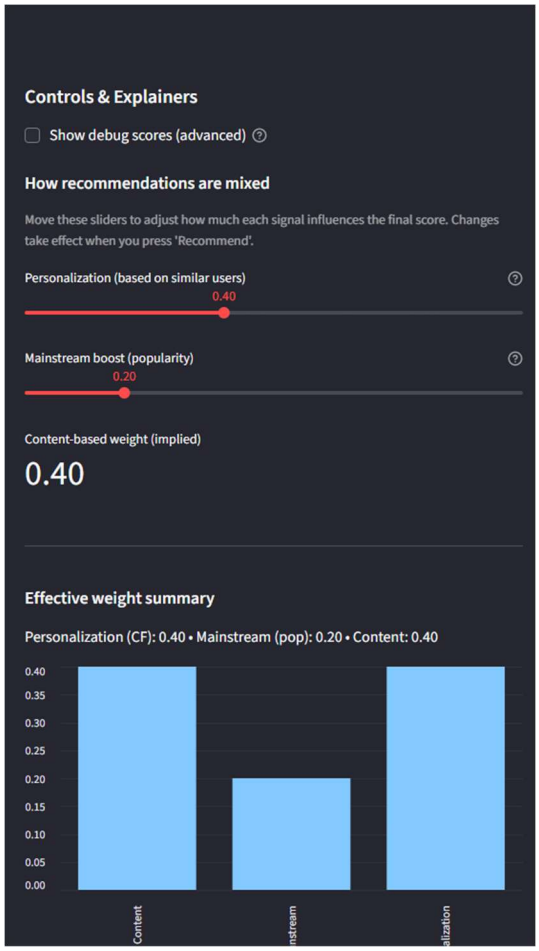


Figure 2. Preferred Weights

The rest of the sidebar can be seen below in **Figure 3**.

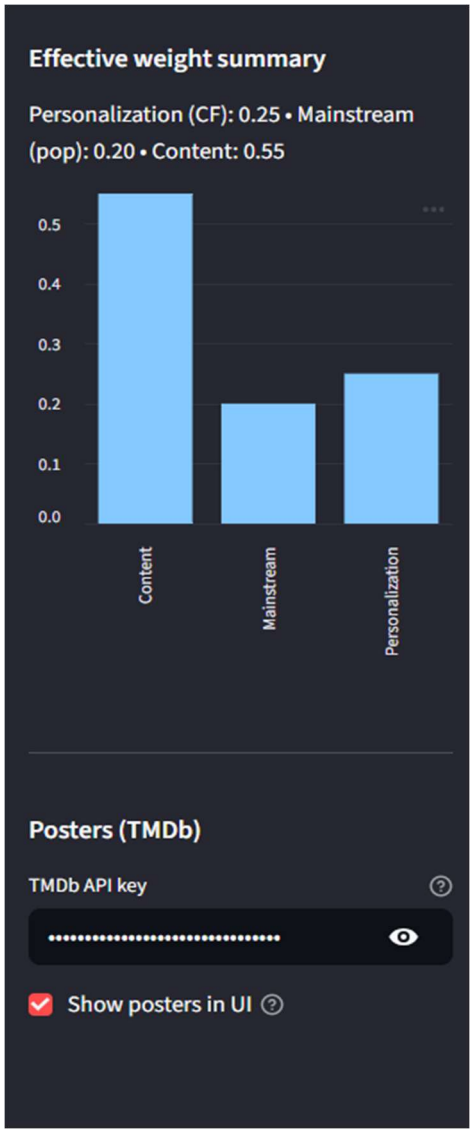


Figure 3. Visual Weight Summary

Figure 4 below shows how the recommended movies are displayed in the dashboard.

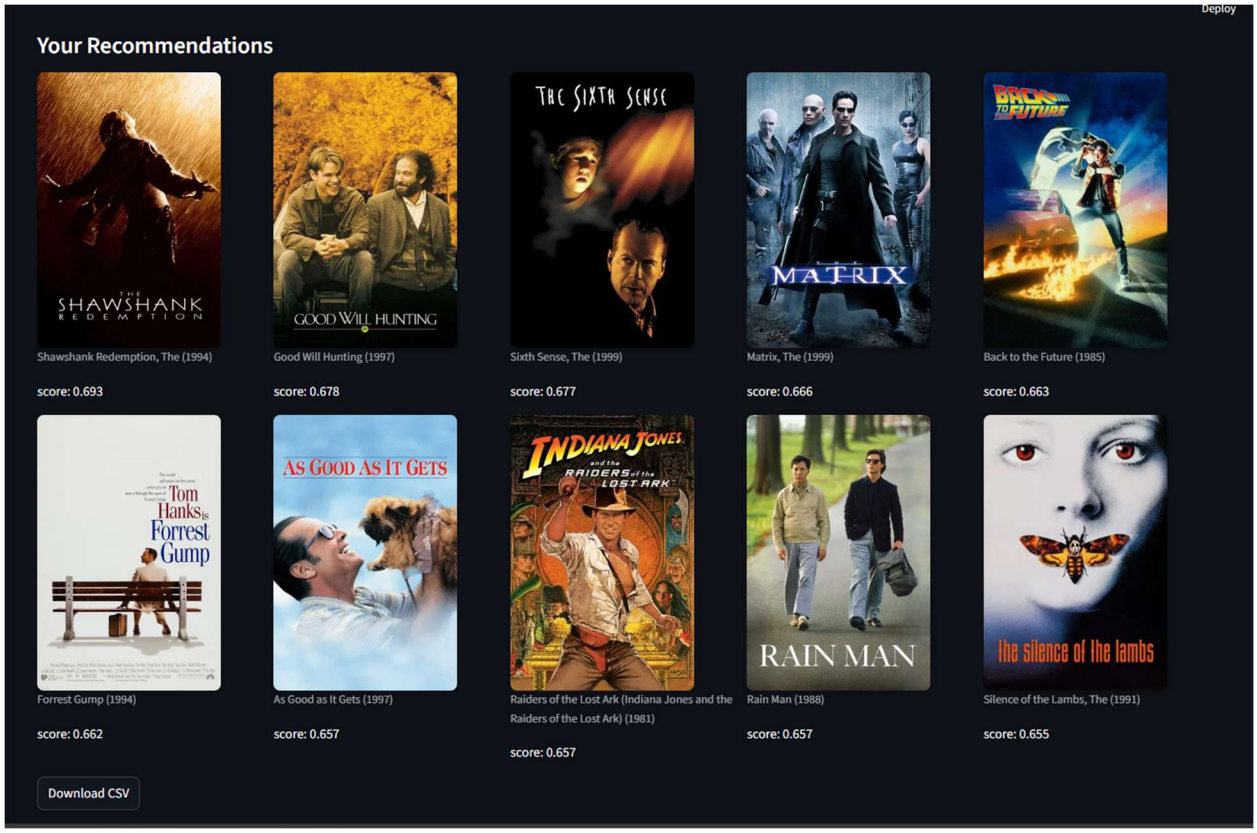


Figure 4. Recommended Movie Interface

4. Conclusions

4.1 How the Dashboard Works

Using the 32M dataset and LensKit, a functional dashboard was able to be made using Streamlit. The overall user interface is intuitive for users and easy to use because users are given 3 random movies to rate on a slider from 0 to 5, as seen in **Figure 1**. If there is a movie a user has not seen, then they are given the option to "swap it" and another new movie will take its place. If a user has not seen *any* of the 5 movies shown for rating, then they have the option to swap all 5 movies at once with the "Shuffle 5 movies" option. There is also a checkbox that can be checked to prefer popular movies if a user would like to receive more popular recommendations, or they can uncheck it to see niche movies that may not be mainstream

Figures 2 and **3** are images of the sidebar on the dashboard where users can adjust their preferences for recommendations by adjusting the weights. So if someone is using the recommender, they want to see movies that are based on users with similar taste as them, they can increase the weight of the "personality" bar, however if a user wants to focus on mainstream movies, they can adjust that bar accordingly. Depending on both the personality and mainstream weights, the amount that movie content affects the recommendations is adjusted automatically.

Then, there is a visual of these weights and how much they contribute to the final score in a bar chart as seen in **Figure 3**. The 32M dataset does not have images for the movie posters, so an API key from the TMBD database was used to link movies and their posters, and the key is also available in the sidebar.

Finally, after a user rates all 5 movies, they can press the "Click here to see recommendations" button to receive 10 movies that the recommender believes they would like based on their ratings and preferred weights, which can be seen in **Figure 4**.

4.2 Comparison to Existing Recommender Systems

The movie recommender system made for Case Study III can be reviewed alongside other current popular recommender systems.

4.2.1 Comparison to Netflix

The Netflix recommender system works by estimating the likelihood that a user will enjoy a movie/show based on viewing history, how a user rates other titles, users with similar tastes, and content information. These factors are like the methods employed to create the recommender for Case Study III, however Netflix also uses some more complex strategies. Netflix takes into account the time-of-day people watch Netflix, the language they watch in, and even how long they watch for. With more demographic data on users, Netflix is able to make very personalized recommendations.

4.2.2 Comparison to Spotify

Like Netflix, Spotify also uses content-based filtering along with collaborative filtering to understand a user's music taste and suggest songs they may like. Spotify takes genre, release date, and even current trends in the music industry to create a unique listening profile for every user. So, the Movie Recommender employs some of the same strategies as Spotify but on a much simpler scale.

5. Limitations

While the Movie Recommender can make predictions on movies a user would like and show 10 movies based on how a user rates 5 given movies, there are still some limitations to the recommender. It relies on simple genres and tags, so it can miss deeper story similarities, and movies with little few ratings are hard to surface. The popularity boost keeps results "safe," but it can tilt too much toward mainstream hits, and this specific collaborative filtering is not ideal at large scale. The way scores are blended, scaling per user and using the same weights for

everyone, means the model mix might fit some people better than others. The diversity step (MMR) uses only content similarity and then mostly preserves the original order, so it reduces duplicates but not as strongly as it could.

6. Recommendations

There are a few ways to improve the Movie Recommender for the future, one way is to make the recommendations much more personalized like how Netflix does it. Adding demographic data for users such as age, gender, first language, and even religion could make it so that the Movie Recommender can suggest very personalized movies and create more connections between similar users. Deploying the dashboard to make it publicly available is also a great next step to taking the Movie Recommender to the next level. Also, saving a user's profile so they can log on with their unique individual logins and save their rating progress, like in IMDB would be a good way to improve the Recommender. Another feature that could improve the usability of the Recommender is perhaps adding an option for users to mark a movie as 'already watched' so it does not show up in recommendations.

7. References

Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets. *ACM Transactions on Interactive Intelligent Systems*, *5*(4), 1–19. https://doi.org/10.1145/2827872

Ekstrand, M. D. (2020, October). Lenskit for python: Next-generation software for recommender systems experiments. In *Proceedings of the 29th ACM international conference on information & knowledge management* (pp. 2999-3006).

Netflix. (n.d.). *How Netflix's Recommendations System Works*. Netflix Help Center. https://help.netflix.com/en/node/100639

Spotify. (2024). *Understanding recommendations on Spotify*. Spotify.com. https://www.spotify.com/us/safetyandprivacy/understanding-recommendations